

Notatki z wykładu: Algorytmy zachłanne

Kody Huffmana

8 kwietnia 2026

1 Motywacja: kodowanie znaków

Często znaki są reprezentowane przy użyciu **8 bitów** (np. kod ASCII), czyli przy pomocy słów kodowych o *stałej* długości. Można jednak kodować oszczędniej, jeśli zezwolimy na słowa kodowe o *zmiennej* długości.

Pomysł: krótkie sekwencje bitów lepiej reprezentują znaki *występujące często*, natomiast znaki *występujące rzadko* reprezentujemy przy użyciu dłuższych sekwencji. W ten sposób minimalizujemy łączną liczbę bitów potrzebnych do zakodowania pliku.

2 Przykład wprowadzający

Rozważmy plik składający się z 58 znaków, w którym każdy znak jest elementem zbioru

$$\{a, e, i, s, t, r, n\}.$$

Częstości wystąpień poszczególnych znaków oraz trzy różne kody zebrano w tabeli:

		<i>a</i>	<i>e</i>	<i>i</i>	<i>s</i>	<i>t</i>	<i>r</i>	<i>n</i>
częstość (liczba wystąpień)		10	15	12	3	4	13	1
kod 1	stała długość	000	001	010	011	100	101	110
kod 2	zmienna długość	001	01	10	00000	0001	11	00001
kod 3	zmienna długość	001	01	10	00010	0001	11	00001

Suma częstości wynosi $10 + 15 + 12 + 3 + 4 + 13 + 1 = 58$, zgodnie z długością pliku.

Kod o stałej długości (kod 1)

Każdy znak zajmuje 3 bity, więc zakodowany plik ma rozmiar

$$3 \cdot 58 = 174 \text{ bity.}$$

Kod o zmiennej długości (kod 2)

Rozmiar pliku to suma iloczynów częstości znaku i długości jego słowa kodowego:

$$10 \cdot 3 + 15 \cdot 2 + 12 \cdot 2 + 3 \cdot 5 + 4 \cdot 4 + 13 \cdot 2 + 1 \cdot 5 = 146 \text{ bitów.}$$

Kod o zmiennej długości pozwala więc zaoszczędzić $174 - 146 = 28$ bitów względem kodu o stałej długości.

3 Kody prefiksowe

Jeśli używamy kodu o zmiennej długości słowa kodowego, to chcemy, aby spełniony był warunek gwarantujący *jednoznaczność dekodowania*.

Definicja 1 (kod prefiksowy). Kod nazywamy **prefiksowym**, jeśli żadne słowo kodowe nie jest prefiksem innego słowa kodowego.¹

Tak długo jak warunek prefiksowości jest spełniony, ciąg bitów daje się odkodować jednoznacznie.

Dlaczego kod 3 jest zły

Kod 3 *nie* jest prefiksowy: słowo kodowe znaku t równe 0001 jest prefiksem słowa kodowego znaku s równego 00010. Rozważmy ciąg bitów

0001001.

Ciąg ten można odkodować na (co najmniej) dwa sposoby:

$$\underbrace{00010}_s \underbrace{01}_e \quad \text{albo} \quad \underbrace{0001}_t \underbrace{001}_a.$$

Nie możemy zatem jednoznacznie odkodować pliku — właśnie dlatego, że 0001 jest prefiksem 00010.

4 Reprezentacja drzewowa kodów

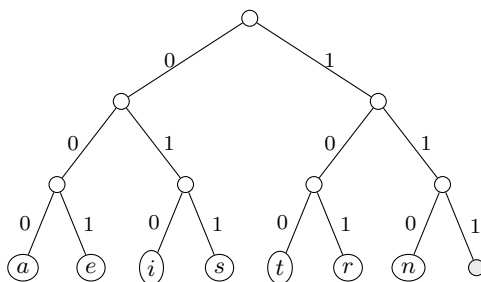
Każdy kod możemy przedstawić jako **drzewo binarne**: schodząc w lewo dopisujemy bit 0, a w prawo bit 1. Symbol jest wówczas ciągiem bitów odczytanym na ścieżce od korzenia (ścieżce wyszukiwania).

Uwaga. Kod jest prefiksowy $\iff$ każdy symbol w reprezentacji drzewowej znajduje się w *liściu*.

Wierzchołki etykietujemy w następujący sposób: każdy wierzchołek v dostaje etykietę równą sumie częstości wystąpień znaków reprezentowanych przez liście poddrzewa o korzeniu v .

Kod 1 — o stałej długości słowa kodowego

Wszystkie liście leżą na tej samej głębokości 3:

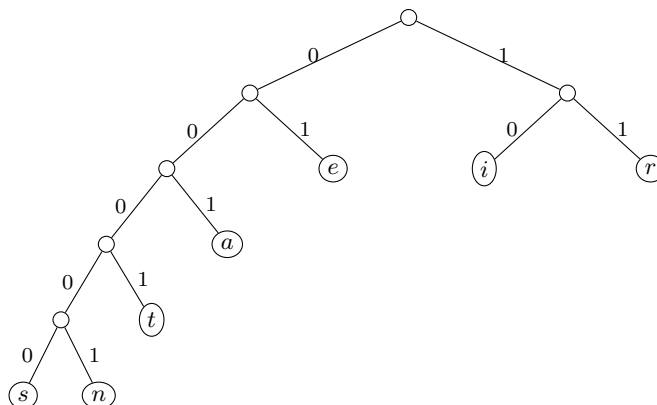


(szary wierzchołek odpowiada nieużywanemu słowu 111).

¹Poprawniejszą (choć rzadziej używaną) nazwą byłoby *kod bezprefiksowy* — nazwa „prefiksowy” utrwalila się umownie.

Kod 2 — o zmiennej długości słowa kodowego

Tu znaki o większej częstości (e, i, r) leżą płycej, a znaki rzadkie (s, n, t) — głębiej:



5 Koszt kodu i optymalność

Niech $\mathbb{C}$ będzie **alfabetem** (zbiorem znaków), a f funkcją częstości

$$f : \mathbb{C} \rightarrow \mathbb{N}, \quad f(c) = \text{częstość znaku } c \in \mathbb{C}.$$

Dla drzewa T reprezentującego kod oznaczmy przez

$$d_T(c) = \text{głębokość liścia reprezentującego znak } c \text{ w drzewie } T$$

długość słowa kodowego znaku c w kodzie reprezentowanym przez drzewo T . Wtedy liczba bitów w zakodowanym pliku wynosi

$$b_T = \sum_{c \in \mathbb{C}} d_T(c) f(c).$$

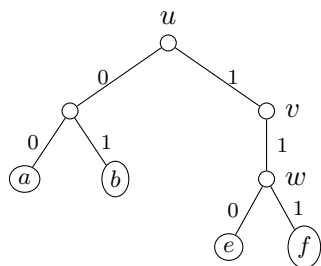
Definicja 2 (kod optymalny). Kod prefiksowy reprezentowany przez drzewo T nazywamy **optymalnym**, jeśli liczba b_T jest najmniejsza możliwa.

Definicja 3 (drzewo binarne i regularne). Drzewo ukorzenione nazywamy **binarnym**, jeśli każdy wierzchołek ma co najwyżej dwoje dzieci. Drzewo binarne nazywamy **regularnym**, jeśli każdy wierzchołek wewnętrzny ma dokładnie dwoje dzieci.

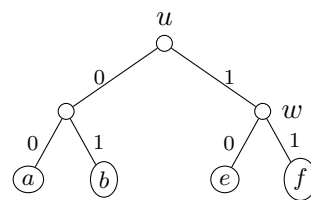
6 Drzewo optymalnego kodu jest regularne

Twierdzenie 1. *Drzewo binarne reprezentujące optymalny kod jest regularne.*

Dowód. Niech T będzie drzewem binarnym reprezentującym optymalny kod. Załóżmy, że *nie* jest ono regularne. Wtedy istnieje wierzchołek wewnętrzny v , który ma dokładnie jedno dziecko w . Jeśli v nie jest korzeniem, to niech u będzie rodzicem wierzchołka v .



T



T'

Niech T' będzie drzewem powstałym z T przez usunięcie v i połączenie u z w (a jeśli v był korzeniem — przez przyjęcie w za nowy korzeń). W T' głębokości wszystkich liści poddrzewa zakorzenionego w w zmalały o 1, a pozostałe liście nie zmieniły głębokości. Zatem

$$b(T') < b(T),$$

co przeczy optymalności T . Otrzymana sprzeczność dowodzi tezy. $\square$

7 Algorytm Huffmana

Algorytm Huffmana konstruuje drzewo reprezentujące optymalny kod prefiksowy dla danego wejścia: alfabetu $\mathbb{C}$ oraz funkcji częstości f .

Idea. Zaczynamy od $|\mathbb{C}|$ wierzchołków izolowanych, które staną się liśćmi, i wykonujemy $|\mathbb{C}| - 1$ operacji *scalania*, zaczynając od najrzadszych znaków. Znaki o najmniejszej częstości trafiają wówczas najgłębiej w drzewie.

Struktura danych. Będziemy używać **kolejki priorytetowej** q z kluczem $f(c)$. Kolejkę priorytetową możemy zaimplementować przy użyciu *kopca binarnego*. Wtedy koszty operacji są następujące:

$$\underbrace{\text{INSERT}}_{O(\log n)}, \quad \underbrace{\text{EXTRACT-MIN}}_{O(\log n)}, \quad \underbrace{\text{CREATE}}_{O(n)}$$

gdzie $n = |\mathbb{C}|$.

Oznaczenia w pseudokodzie: z — nowo tworzony wierzchołek, $left(z)$ i $right(z)$ — jego dzieci, q — kolejka priorytetowa.

Algorytm konstruuje drzewo binarne za pomocą $n-1$ operacji scalania, w którym każdy wierzchołek wewnętrzny ma dokładnie dwoje dzieci (drzewo jest regularne).

8 Przykład działania algorytmu

Wracamy do alfabetu $\{a, e, i, s, t, r, n\}$ z częstościami z tabeli. Algorytm w kolejnych krokach scala dwa elementy o najmniejszych kluczach:

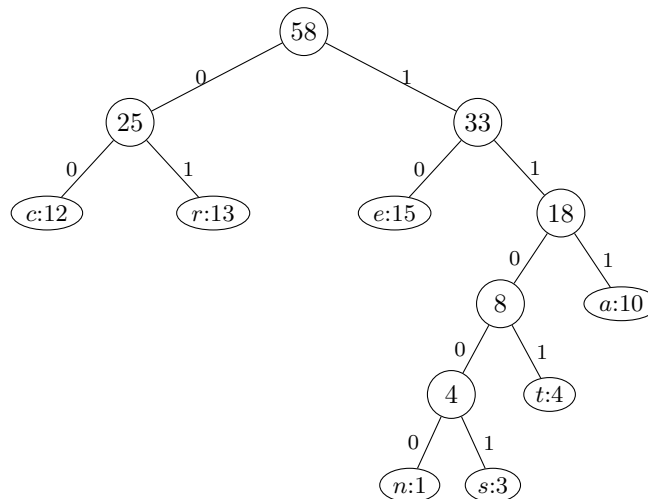
Algorithm 1 Huffman($\mathbb{C}, f$)

```
1:  $n \leftarrow |\mathbb{C}|$  {liczba znaków;  $O(n)$ }
2: utwórz kolejkę  $q$  na podstawie  $f$  { $O(n)$ }
3: for  $i \leftarrow 1$  to  $n - 1$  do
4:   utwórz nowy wierzchołek  $z$  {pętla wykonuje się  $n - 1$  razy}
5:    $x \leftarrow \text{left}(z) \leftarrow \text{EXTRACT-MIN}(q)$  { $O(\log n)$ }
6:    $y \leftarrow \text{right}(z) \leftarrow \text{EXTRACT-MIN}(q)$  { $O(\log n)$ }
7:    $f(z) \leftarrow f(x) + f(y)$ 
8:    $\text{INSERT}(q, z)$  { $O(\log n)$ }
9: end for
10: return  $\text{EXTRACT-MIN}(q)$  {korzeń drzewa}
```

krok	scalane elementy	nowy wierzchołek
0	— (start)	$n:1, s:3, t:4, a:10, c:12, r:13, e:15$
1	$n:1$ oraz $s:3$	4
2	4 oraz $t:4$	8
3	8 oraz $a:10$	18
4	$c:12$ oraz $r:13$	25
5	18 oraz $e:15$	33
6	25 oraz 33	58 (korzeń)

Uwaga. W tym przykładzie częstość $c:12$ przyjęto dodatkowo (rozszerzony zestaw znaków z wykładu); kluczowe jest, że w każdym kroku scalamy dwa najmniejsze klucze.

Drzewo końcowe



Wynikowe słowa kodowe

Odczytując ścieżki od korzenia (0 w lewo, 1 w prawo) otrzymujemy:

znak	słowo kodowe	długość
<i>c</i>	00	2
<i>r</i>	01	2
<i>e</i>	10	2
<i>a</i>	111	3
<i>t</i>	1101	4
<i>n</i>	11000	5
<i>s</i>	11001	5

Łączna liczba bitów dla tego kodu:

$$12 \cdot 2 + 13 \cdot 2 + 15 \cdot 2 + 10 \cdot 3 + 4 \cdot 4 + 1 \cdot 5 + 3 \cdot 5 = 146 \text{ bitów},$$

czyli tyle samo, co dla kodu 2 — oba kody są optymalne.

9 Złożoność czasowa

Pętla wykonuje się $n - 1$ razy, a jej wnętrze (dwie operacje EXTRACT-MIN oraz jedna INSERT) kosztuje $O(\log n)$. Stąd

$$(n - 1) \cdot O(\log n) = O(n \log n).$$

Wraz z kosztem inicjalizacji $O(n)$ całkowita złożoność algorytmu Huffmana wynosi

$O(n \log n).$